

Deep Learning for Data Science

DS 542

<https://dl4ds.github.io/fa2025/>

Transformer Details

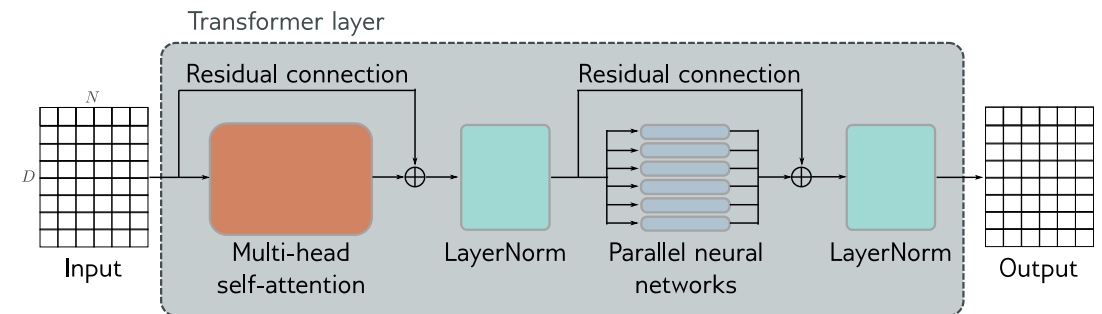
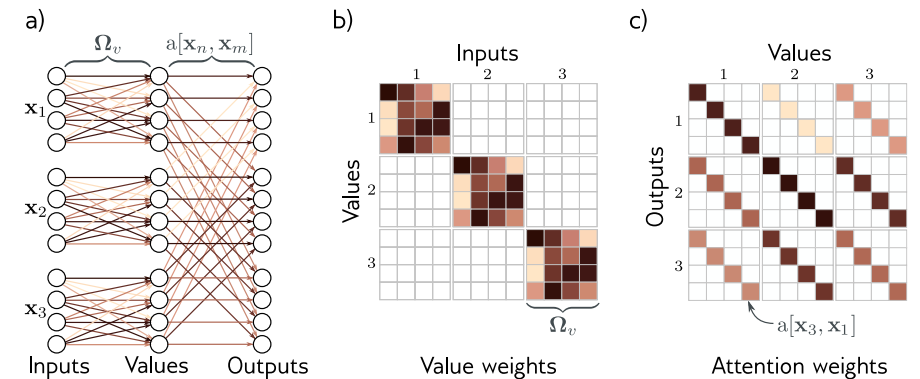
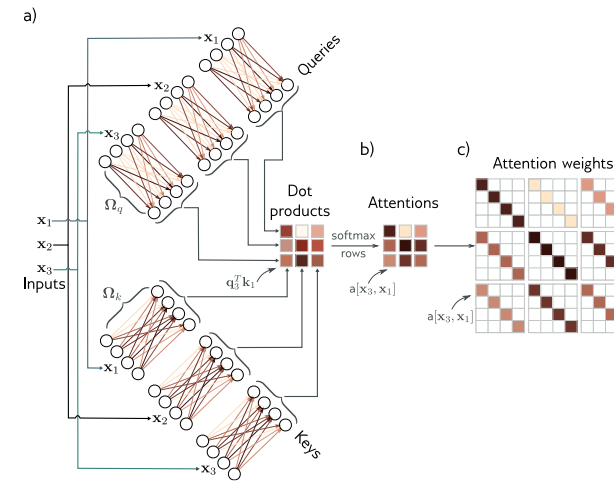


Plan for Today

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences

Recap From Part 1

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models
 - Encoder
 - Decoder
 - Encoder-Decoder



3 Types of Transformer Models

1. *Encoder* – transforms text embeddings into representations that support variety of tasks (e.g. sentiment analysis, classification)
❖ Model Example: BERT
2. *Decoder* – predicts the next token to continue the input text (e.g. ChatGPT, AI assistants)
❖ Model Example: GPT4, GPT4
3. *Encoder-Decoder* – used in sequence-to-sequence tasks, where one text string is converted to another (e.g. machine translation)

Any Questions?

???

Moving on

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences

What's a Token?

A small chunk of text that we use to aid language modeling.

- Represents one or more bytes
- Input texts are greedily divided into tokens.
 - Longest prefix matching a token.
- Token set also constructed greedily.
 - Start with 256 possible bytes.
 - Then greedily pick the most common pairs of adjacent tokens.

Why Tokens?

Instead of...

- Bits - not enough semantics* and missing intrabyte positioning
- Bytes - not enough semantics* for Unicode
- Characters - too many of them if we try to support all languages
- Words - even more words than characters

Remember:

- One-hot/Softmax tactic means we will have at least one output per possible output value, and many more parameters in practice.

Unicode Standard and UTF-8

- **Unicode** – *variable length* character encoding standard. currently defines 149,813 characters and 161 scripts, including emoji, symbols, etc.
- **Unicode Codepoint** – can represent up to $17 \times 2^{16} = 1,114,112$ entries. e.g. U+0000 – U+10FFFF in hexadecimal
- **Unicode Transformation Standard (e.g. UTF-8)** – is a *variable length encoding* using one to four bytes
 - First 128 chars same as ASCII

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxxxxx			
U+0080	U+07FF	110xxxxx	10xxxxxx		
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
U+10000	^[b] U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Covers ASCII

Covers remainder of almost all Latin-script alphabets

Basic Multilingual Plane including Chinese, Japanese and Korean characters

Emoji, historic scripts, math symbols

<https://en.wikipedia.org/wiki/Unicode>

<https://en.wikipedia.org/wiki/UTF-8>

Example Tokens

```
▶ import tiktoken
```

```
[6] enc = tiktoken.encoding_for_model("gpt-4o")
```

```
▶ for i in range(1024):  
    d = enc.decode([i])  
    if len(d) >= 4:  
        print(i, enc.decode([i]))
```

```
257  
269  
290 the  
309  
326 and  
352  
387 ation  
395 for  
406 con  
408  
440 pro  
447 port  
452 com  
475 ction  
481 you  
483 with  
484 that  
495 this  
506  
508 ment  
518 ----  
529 turn  
530  
553 are  
561 import  
562 able  
583 ight  
584 ublic  
591 from  
595 ****  
600 tring  
620 new  
622 return  
623 The  
625 not  
626  
634 your  
661 que  
665 can  
673 was  
677 int  
679 have  
686 par  
694 res  
699  
700 form  
717 get  
722 all  
728 ject  
731 des  
735 alue  
738 will  
740 ();  
744 class  
751 public  
756 ions  
758 }  
763 -----  
766 ance  
767 ould  
773 ient  
775 .get
```

Tokenizer



Tokenizer chooses input “units”, e.g. words, sub-words, characters via *tokenizer training*

In tokenizer training, commonly occurring substrings are greedily merged based on their frequency, starting with character pairs

Tokenization Issues

“A lot of the issues that may look like issues with the neural network architecture actually trace back to tokenization. Here are just a few examples” – Andrej Karpathy

- Why can't LLM spell words? Tokenization.
- Why can't LLM do super simple string processing tasks like reversing a string? Tokenization.
- Why is LLM worse at non-English languages (e.g. Japanese)? Tokenization.
- Why is LLM bad at simple arithmetic? Tokenization.
- Why did GPT-2 have more than necessary trouble coding in Python? Tokenization.
- Why did my LLM abruptly halt when it sees the string "<|endoftext|>"? Tokenization.
- What is this weird warning I get about a "trailing whitespace"? Tokenization.
- Why did the LLM break if I ask it about "SolidGoldMagikarp"? Tokenization.
- Why should I prefer to use YAML over JSON with LLMs? Tokenization.
- Why is LLM not actually end-to-end language modeling? Tokenization.
- What is the real root of suffering? Tokenization.

<https://github.com/karpathy/minbpe/blob/master/lecture.md>

Any Questions?

???

Moving on

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences

Tokenization Matters

From the gpt-4o announcement,

“It matches GPT-4 Turbo performance on text in English and code, with significant improvement on text in non-English languages, while also being much faster and 50% cheaper in the API.”

Gains were from increasing the number of tokens in the updated tokenizer.

<https://openai.com/index/hello-gpt-4o/>

Russian 1.7x fewer tokens (from 39 to 23)	Привет, меня зовут GPT-4o. Я — новая языковая модель, приятно познакомиться!
Korean 1.7x fewer tokens (from 45 to 27)	안녕하세요, 제 이름은 GPT-4o입니다. 저는 새로운 유형의 언어 모델입니다, 만나서 반갑습니다!
Vietnamese 1.5x fewer tokens (from 46 to 30)	Xin chào, tên tôi là GPT-4o. Tôi là một loại mô hình ngôn ngữ mới, rất vui được gặp bạn!
Chinese 1.4x fewer tokens (from 34 to 24)	你好，我的名字是GPT-4o。我是一种新型的语言模型，很高兴见到你!
Japanese 1.4x fewer tokens (from 37 to 26)	こんにちは、私の名前はGPT-4oです。私は新しいタイプの言語モデルです。初めまして！

Tokenizer

Two common tokenizers:

- Byte Pair Encoding (BPE) – Used by OpenAI GPT2, GPT4, etc.
 - The BPE algorithm is "byte-level" because it runs on UTF-8 encoded strings.
 - This algorithm was popularized for LLMs by the [GPT-2 paper](#) and the associated GPT-2 [code release](#) from OpenAI. [Sennrich et al. 2015](#) is cited as the original reference for the use of BPE in NLP applications. Today, all modern LLMs (e.g. GPT, Llama, Mistral) use this algorithm to train their tokenizers.*
- sentencepiece
 - (e.g. Llama, Mistral) use [sentencepiece](#) instead. Primary difference being that sentencepiece runs BPE directly on Unicode code points instead of on UTF-8 encoded bytes.

* <https://github.com/karpathy/minbpe/tree/master>

BPE Pseudocode

Initialize vocabulary with individual characters in the text and their frequencies

While desired vocabulary size not reached:

Identify the most frequent pair of adjacent tokens/characters in the vocabulary

Merge this pair to form a new token

Update the vocabulary with this new token

Recalculate frequencies of all tokens including the new token

Return the final vocabulary

Enforce a Token Split Pattern

```
GPT2_SPLIT_PATTERN = r''''(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:^\s\p{L}\p{N}+|\s+(?!S)|\s+''''
```

```
GPT4_SPLIT_PATTERN = r''''(?:i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?+\p{L}+|\p{N}{1,3}|  
?(?:^\s\p{L}\p{N}++[\r\n]*|\s*[\r\n]|\s+(?!S)|\s+''''
```

- Do not allow tokens to merge across certain characters or patterns
- Common contraction endings: ‘ll, ‘ve, ‘re
- Match words with a leading space
- Match numeric sequences
- carriage returns, new lines

GPT4 Tokenizer

Tiktokenizer

cl100k_base is the GPT4

tokenizer

cl100k_base

a sailor went to sea sea sea
to see what he could see see see
but all that he could see see see
was the bottom of the deep blue sea sea sea

Token count

36

a·sailor·went·to·sea·sea·sea\n
to·see·what·he·could·see·see·see\n
but·all·that·he·could·see·see·see\n
was·the·bottom·of·the·deep·blue·sea·sea·sea

[64, 93637, 4024, 311, 9581, 9581, 9581, 198, 99
8, 1518, 1148, 568, 1436, 1518, 1518, 1518, 198,
8248, 682, 430, 568, 1436, 1518, 1518, 1518, 198,
16514, 279, 5740, 315, 279, 5655, 6437, 9581, 958
1, 9581]

☒ Show whitespace

<https://tiktokenizer.vercel.app/>

GPT2 Tokenizer

Tiktokenizer

```
class Tokenizer:
    """Base class for Tokenizers"""

    def __init__(self):
        # default: vocab size of 256 (all bytes), no merges,
        # no patterns
        self.merges = {} # (int, int) -> int
        self.pattern = "" # str
        self.special_tokens = {} # str -> int, e.g.
        {'<|endoftext|>': 100257}
        self.vocab = self._build_vocab() # int -> bytes
```

You can see some issues with the GPT2 tokenizer with respect to python code

<https://tiktokenizer.vercel.app/>

gpt2

Token count
146

```
class Tokenizer:\n    ...\"\"\"Base class for Tokenizers\"\"\"\n\n    \n    ...def __init__(self):\n        ...# default: vocab size of 256 (all bytes), no m\n        erges, no patterns\n        ...self.merges = {} # (int, int) -> int\n        ...self.pattern = \"\" # str\n        ...self.special_tokens = {} # str -> int, e.g. \n        {'<|endoftext|>': 100257}\n        ...self.vocab = self._build_vocab() # int -> byte\n        s
```

```
[4871, 29130, 7509, 25, 198, 220, 220, 220, 37227, 148\n81, 1398, 329, 29130, 11341, 37811, 628, 220, 220, 22\n0, 825, 11593, 15003, 834, 7, 944, 2599, 198, 220, 22\n0, 220, 220, 220, 220, 1303, 4277, 25, 12776, 39\n7, 2546, 286, 17759, 357, 439, 9881, 828, 645, 4017, 3\n212, 11, 645, 7572, 198, 220, 220, 220, 220, 220, 220,\n220, 2116, 13, 647, 3212, 796, 23884, 1303, 357, 600,\n11, 493, 8, 4613, 493, 198, 220, 220, 220, 220, 220, 2\n20, 220, 2116, 13, 33279, 796, 13538, 1303, 965, 198,\n220, 220, 220, 220, 220, 220, 220, 2116, 13, 20887, 6\n2, 83, 482, 641, 796, 23884, 1303, 965, 4613, 493, 11,\n304, 13, 70, 13, 1391, 6, 50256, 10354, 1802, 28676, 9\n2, 198, 220, 220, 220, 220, 220, 220, 220, 2116, 13, 1\n8893, 397, 796, 2116, 13557, 11249, 62, 18893, 397, 34\n19, 1303, 493, 4613, 9881]
```

☒ Show whitespace

GPT4 Tokenizer

Tiktokenizer

```
class Tokenizer:
    """Base class for Tokenizers"""

    def __init__(self):
        # default: vocab size of 256 (all bytes), no merges,
        # no patterns
        self.merges = {} # (int, int) -> int
        self.pattern = "" # str
        self.special_tokens = {} # str -> int, e.g.
        {'<|endoftext|>': 100257}
        self.vocab = self._build_vocab() # int -> bytes
```

cl100k_base

Token count
96

```
class Tokenizer:\n    ...\"\"\"Base class for Tokenizers\"\"\"\n\n    \n    ...def __init__(self):\n        ...# default: vocab size of 256 (all bytes), no m\n        erges, no patterns\n        ...self.merges = {} # (int, int) -> int\n        ...self.pattern = \"\" # str\n        ...self.special_tokens = {} # str -> int, e.g.\n        {'<|endoftext|>': 100257}\n        ...self.vocab = self._build_vocab() # int -> byte\n        s
```

Issues are improved with GPT4
tokenizer

```
[1058, 9857, 3213, 512, 262, 4304, 4066, 538, 369, 985\n7, 12509, 15425, 262, 711, 1328, 2381, 3889, 726, 997,\n286, 674, 1670, 25, 24757, 1404, 315, 220, 4146, 320,\n543, 5943, 705, 912, 82053, 11, 912, 12912, 198, 286,\n659, 749, 2431, 288, 284, 4792, 674, 320, 396, 11, 52\n8, 8, 1492, 528, 198, 286, 659, 40209, 284, 1621, 674,\n610, 198, 286, 659, 64308, 29938, 284, 4792, 674, 610,\n1492, 528, 11, 384, 1326, 13, 5473, 100257, 1232, 220,\n1041, 15574, 534, 286, 659, 78557, 284, 659, 1462, 595\n7, 53923, 368, 674, 528, 1492, 5943]
```

☒ Show whitespace

<https://tiktokenizer.vercel.app/>

a) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	l	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

Byte Pair Encoding (BPE)

Example

Minimal starting vocabulary of subset of lower case latin alphabet and space `\_`.

a) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	l	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h	l	u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

Byte Pair Encoding (BPE)

Example

Find the most frequent pair of adjacent tokens, `se`, in this case and form new token.

Byte Pair Encoding (BPE) Example

a) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	l	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h	l	u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	se	a	e_	t	o	h	l	u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

Next most frequent pair of tokens is `e\_`

Byte Pair Encoding (BPE) Example

a) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	l	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h	l	u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	se	a	e	t	o	h	l	u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

⋮

⋮

Continue until you hit your vocabulary size limit.

d) see_sea_e_b_l_w_a_could_hat_he_o_t_t_the_to_u_a_d_f_m_n_p_s_sailor_to

see_	sea_	e	b	l	w	a	could_	hat_	he_	o	t	t_	the_	to_	u	a_	d	f	m	n	p	s	sailor_	to
7	6	4	3	3	3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1

Byte Pair Encoding (BPE) Example

a) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h		u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h		u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	se	a	e	t	o	h		u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

⋮ ⋮

d) see_sea_e_b_l_w_a_could_hat_he_o_t_t_the_to_u_a_d_f_m_n_p_s_sailor_to
7 6 4 3 3 3 3 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1

⋮ ⋮ ⋮

e) see_sea_could_he_the_a_all_blue_bottom_but_deep_of_sailor_that_to_was_went_what_
7 6 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

a) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h		u	b	d	w	c	f	i	m	n	p	r	
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h		u	b	d	w	c	s	f	i	m	n	p	r	
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	se	a	e	t	o	h		u	b	d	e	w	c	s	f	i	m	n	p	r	
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1	1

⋮

⋮

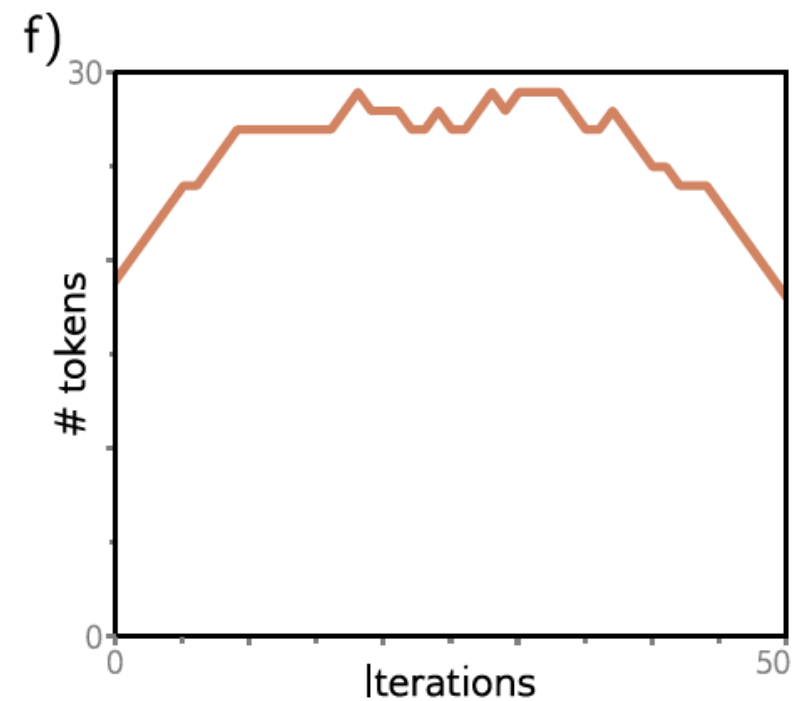
d) see_sea_e_b|w_a_could_hat_he_o_t_t|the_to_u_a_d_f|m_n_p_s_sailor_to
 7 6 4 3 3 3 3 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1

⋮

⋮

⋮

e) see_sea_could_he_the_a_all_blue_bottom_but_deep_of_sailor_that_to_was_went_what_
 7 6 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

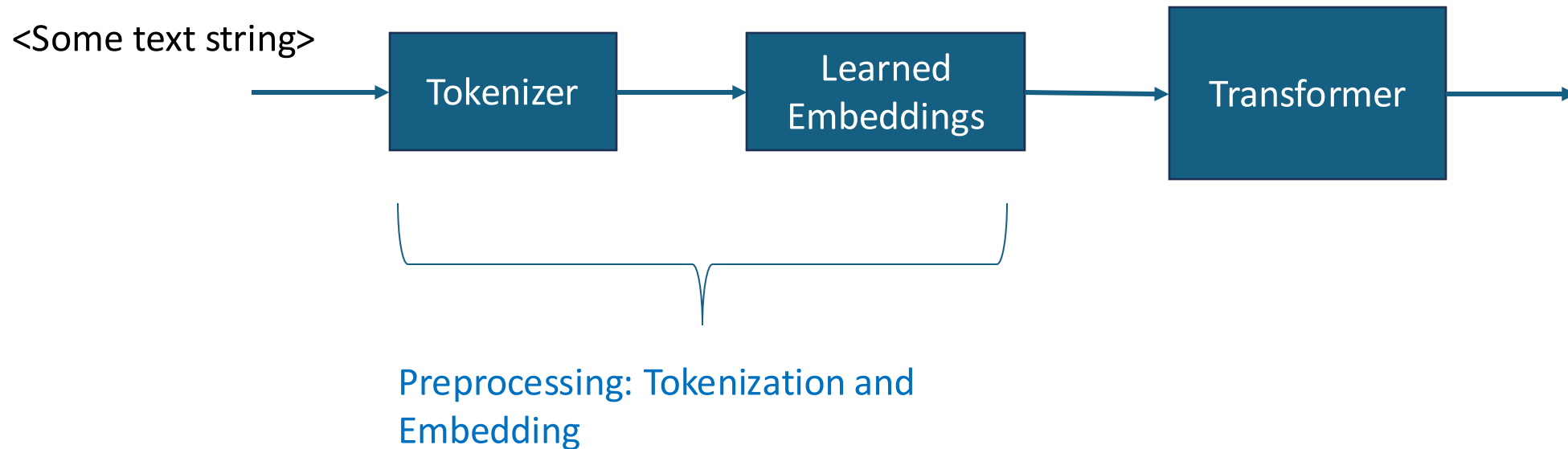


Generally # of tokens increases and then starts decreasing after continuing to merge tokens

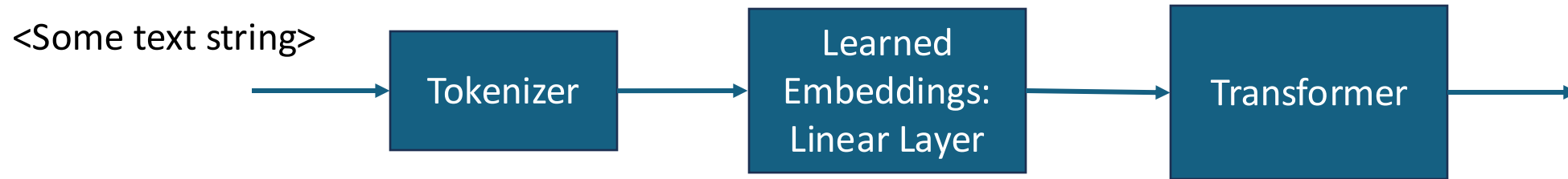
NLP Preprocessing Pipeline

Transformers don't work on character string directly, but rather on vectors.

The character strings must be converted to vectors



Learned Embeddings

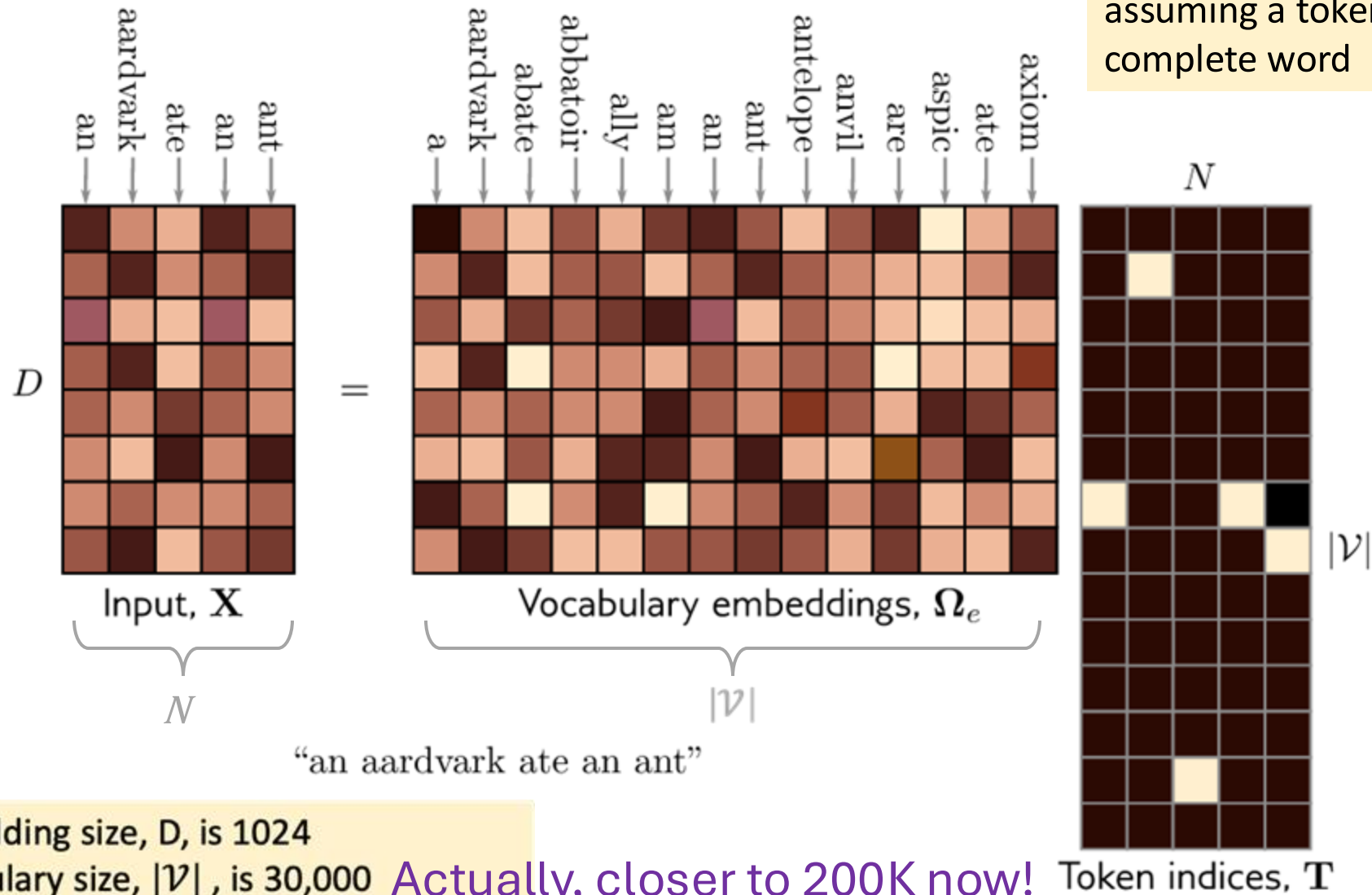


- After the tokenizer, you have an updated “vocabulary” indexed by token ID
- Next step is to translate the token into an embedding vector
- Translation is done via a linear layer which is typically learned with the rest of the transformer model

```
self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

- Special layer definition, likely to exploit sparsity of input

Embeddings Output



- Typical embedding size, D , is 1024
- Typical vocabulary size, $|V|$, is 30,000
- So 30M parameters just for this matrix!

Actually, closer to 200K now!

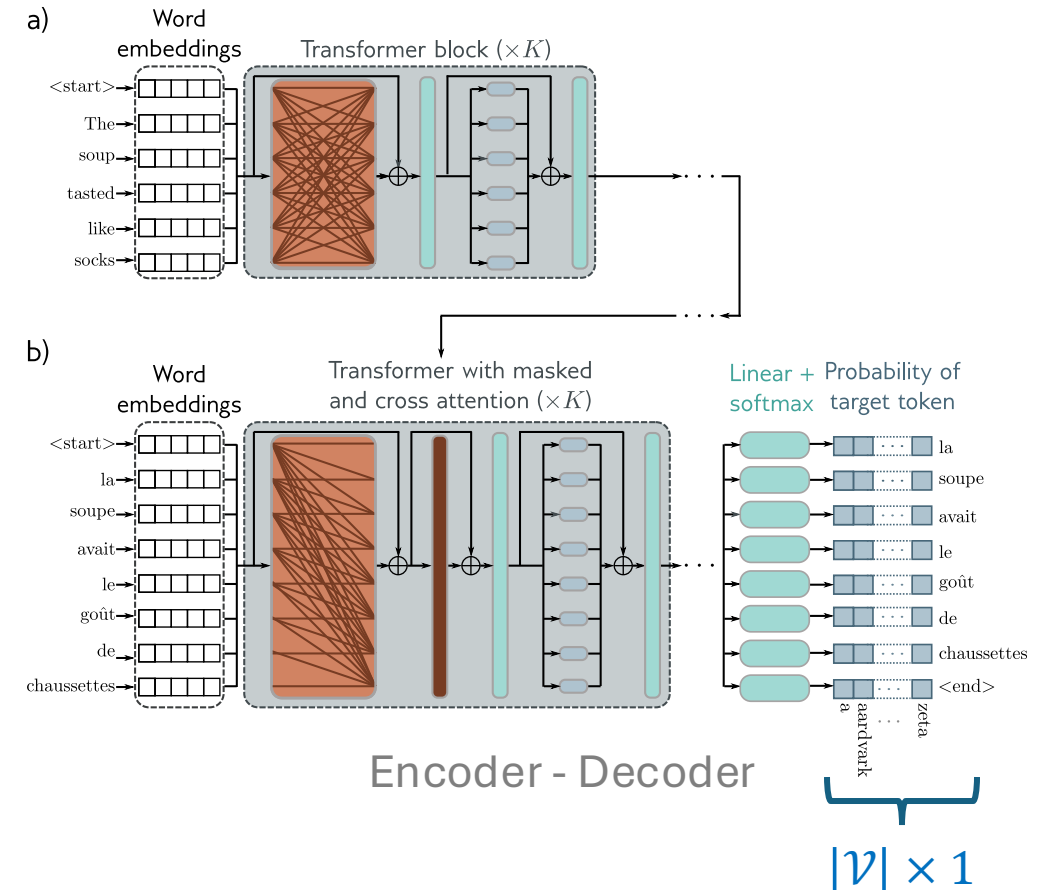
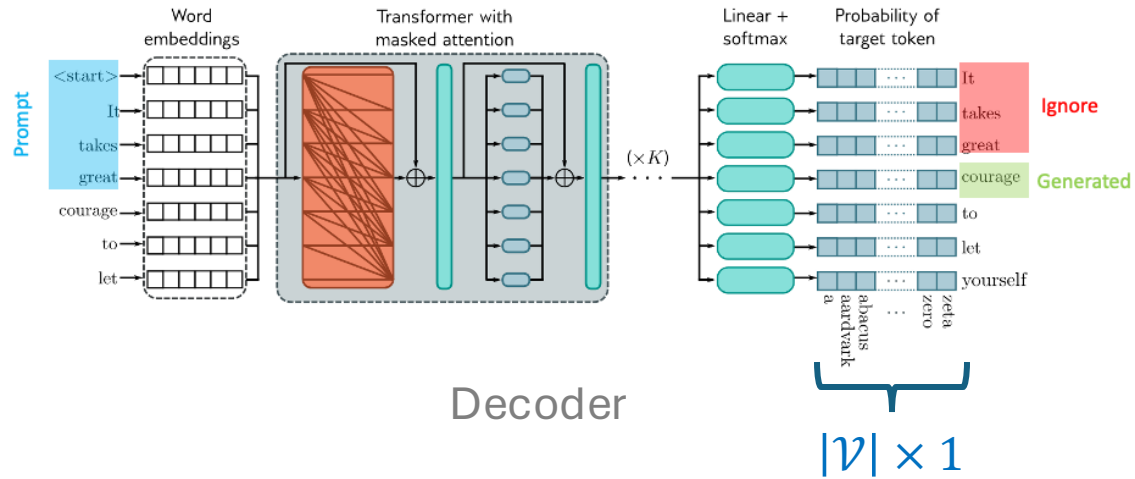
Any Questions?

???

Moving on

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences

Next Token Selection



- Recall: output is a $|\mathcal{V}| \times 1$ vector of probabilities
- How should we pick the next token?
- Trade off between **accuracy** and **diversity**

Next Token Selection

Recall: output is a $|\mathcal{V}| \times 1$ vector of probabilities



Selectin methods:

- Greedy selection
- Top-K
- Nucleus
- Temperature
- Beam search

Next Token Selection – Greedy

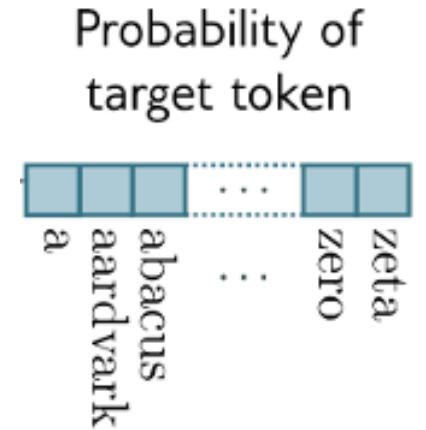
Pick most likely token (greedy)

Simple to implement. Just take the max().

$$\hat{y}_t = \operatorname{argmax}_{w \in \mathcal{V}} [Pr(y_t = w | \hat{\mathbf{y}}_{<t}, \mathbf{x}, \phi)]$$

in PyTorch

```
outputs = model(inputs)
value, index = outputs.max(1)
```

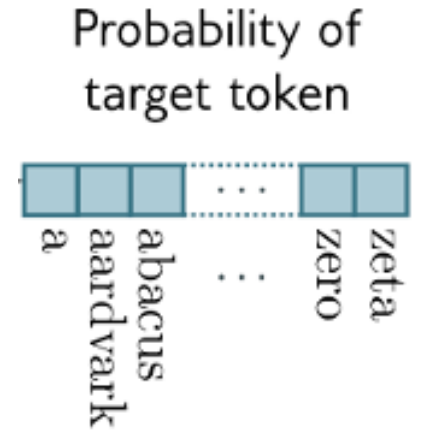


Might pick first token y_0 , but then there is no y_1 where $Pr(y_1 | y_0)$ is high.

Result is generic and predictable. Same output for a given input context.

Next Token Selection -- Sampling

Sample from the probability distribution



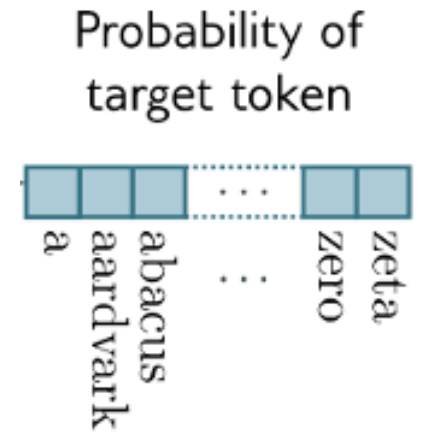
Get a bit more diversity in the output

Will occasionally sample from the long tail of the distribution, producing some unlikely word combinations.

But real text does have unlikely words too.

Next Token Selection – Top K Sampling

1. Generate the probability vector as usual
2. Sort tokens by likelihood
3. Discard all but top k most probable words
4. Renormalize the probabilities to be valid probability distribution (e.g. sum to 1)
5. Sample from the new distribution



Diversifies word selection.

Depends on the distribution. Could be low variance, reducing diversity.

Next Token Selection – Nucleus Sampling

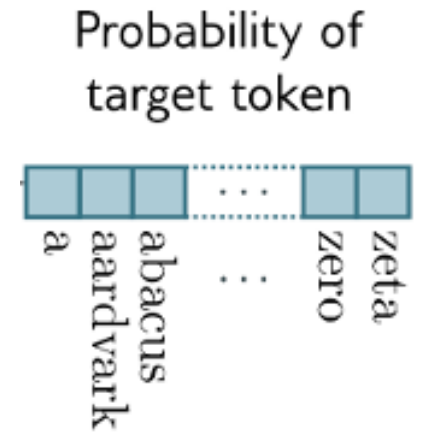
Instead of keeping top- k , keep the top p percent of the probability mass.

Choose from the smallest set from the vocabulary such that

$$\sum_{w \in V(p)} P(w | \mathbf{w}_{<t}) \geq p.$$

Diversifies word selection with less dependence on nature of distribution.

Depends on the distribution. Could be low variance, reducing diversity.



Next Token Selection - Temperature

- Before applying softmax to calculate probabilities, divide the logit outputs by a temperature T .

$$\text{softmax}_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$\text{softmax}_i(\mathbf{z}, T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- What happens as $T \rightarrow \infty$?
- What happens as $T \rightarrow 0$?
- Offered in most LLM interfaces.

Next Token Selection – Beam Search

Commonly used in *machine translation*

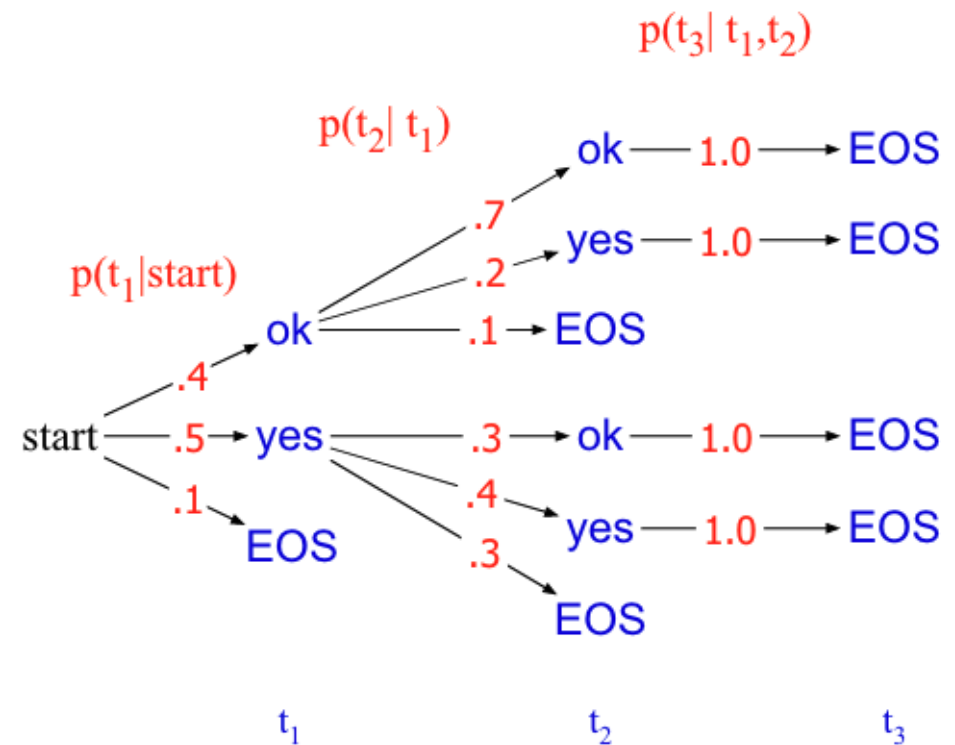
Maintain multiple output choices and then choose best combinations later via tree search

$V = \{\text{yes}, \text{ok}, \text{<eos>}\}$

We want to maximize $p(t_1, t_2, t_3)$.

Greedy: $0.5 \times 0.4 \times 1.0 = 0.20$

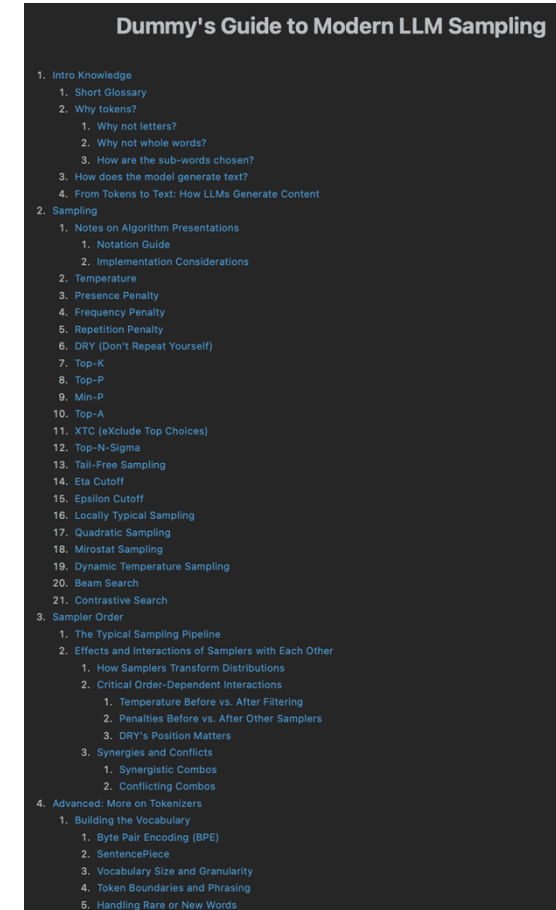
Optimal: $0.4 \times 0.7 \times 1.0 = 0.28$



**TLDR: keep best k paths at each level of tree.
Less popular as models have gotten bigger.**

Dummy's Guide to LLM Sampling

- <https://reentry.co/samplers>



Dummy's Guide to Modern LLM Sampling

- 1. Intro Knowledge
 - 1. Short Glossary
 - 2. Why tokens?
 - 1. Why not letters?
 - 2. Why not whole words?
 - 3. How are the sub-words chosen?
 - 3. How does the model generate text?
 - 4. From Tokens to Text: How LLMs Generate Content
- 2. Sampling
 - 1. Notes on Algorithm Presentations
 - 1. Notation Guide
 - 2. Implementation Considerations
 - 2. Temperature
 - 3. Presence Penalty
 - 4. Frequency Penalty
 - 5. Repetition Penalty
 - 6. DRY (Don't Repeat Yourself)
 - 7. Top-K
 - 8. Top-P
 - 9. Min-P
 - 10. Top-A
 - 11. XTC (eXclude Top Choices)
 - 12. Top-N-Sigma
 - 13. Tail-Free Sampling
 - 14. Eta Cutoff
 - 15. Epsilon Cutoff
 - 16. Locally Typical Sampling
 - 17. Quadratic Sampling
 - 18. Mirostat Sampling
 - 19. Dynamic Temperature Sampling
 - 20. Beam Search
 - 21. Contrastive Search
- 3. Sampler Order
 - 1. The Typical Sampling Pipeline
 - 2. Effects and Interactions of Samplers with Each Other
 - 1. How Samplers Transform Distributions
 - 2. Critical Order-Dependent Interactions
 - 1. Temperature Before vs. After Filtering
 - 2. Penalties Before vs. After Other Samplers
 - 3. DRY's Position Matters
 - 3. Synergies and Conflicts
 - 1. Synergistic Combos
 - 2. Conflicting Combos
- 4. Advanced: More on Tokenizers
 - 1. Building the Vocabulary
 - 1. Byte Pair Encoding (BPE)
 - 2. SentencePiece
 - 3. Vocabulary Size and Granularity
 - 4. Token Boundaries and Phrasing
 - 5. Handling Rare or New Words

Any Questions?

???

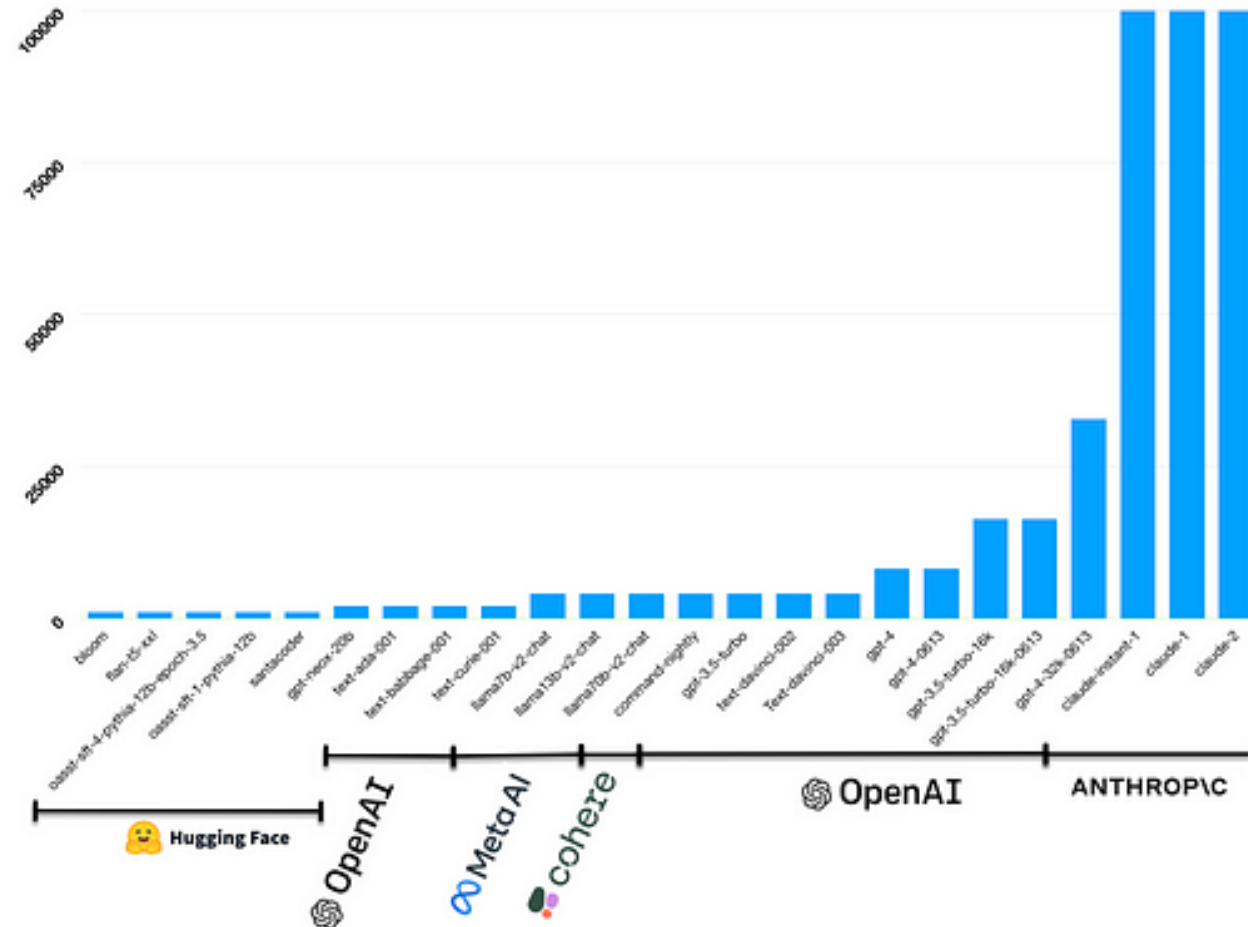
Moving on

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences

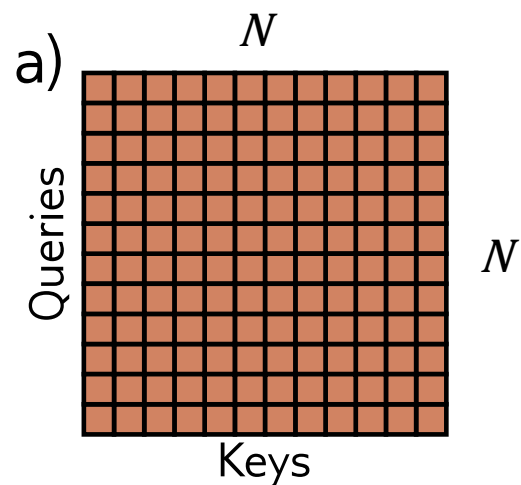
Context Length of LLMs

Large Language Model Context Size

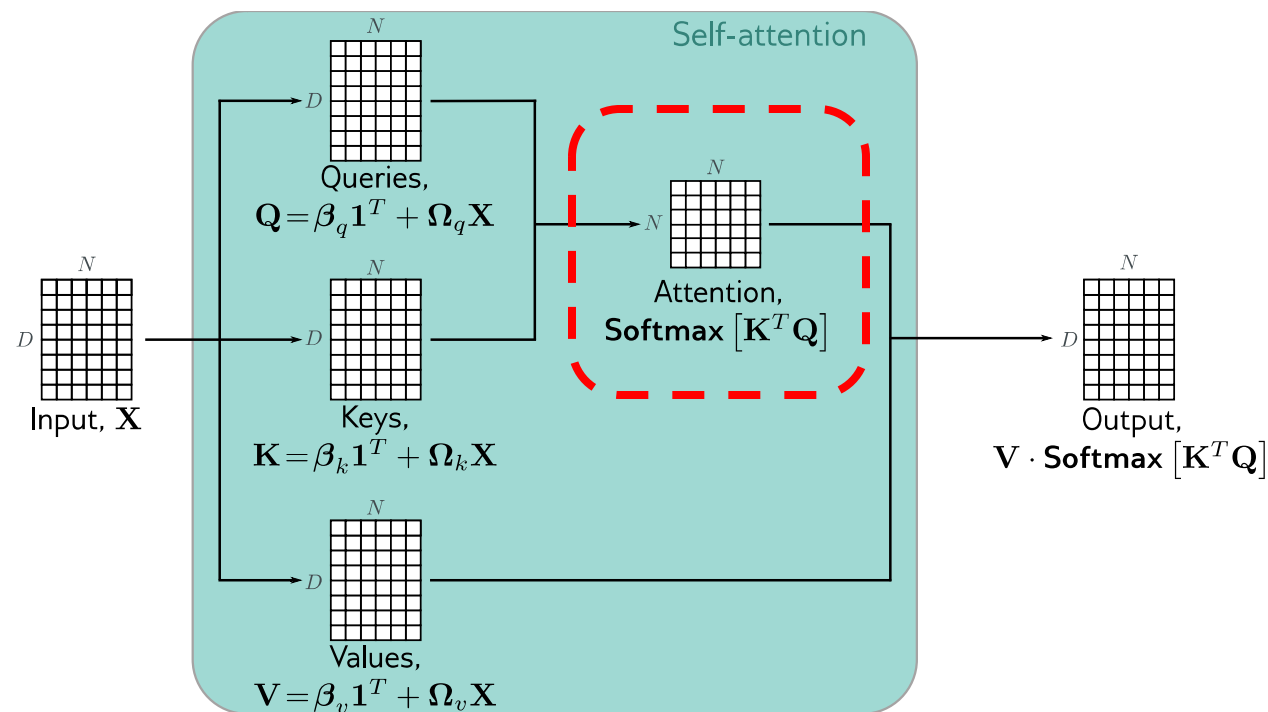
Model	Context Length
Llama 2	32K
GPT4	32K
GPT-4 Turbo, Llama 3.1	128K
Claude 3.5 Sonnet	200K
Google Gemini 1.5 Pro	Millions



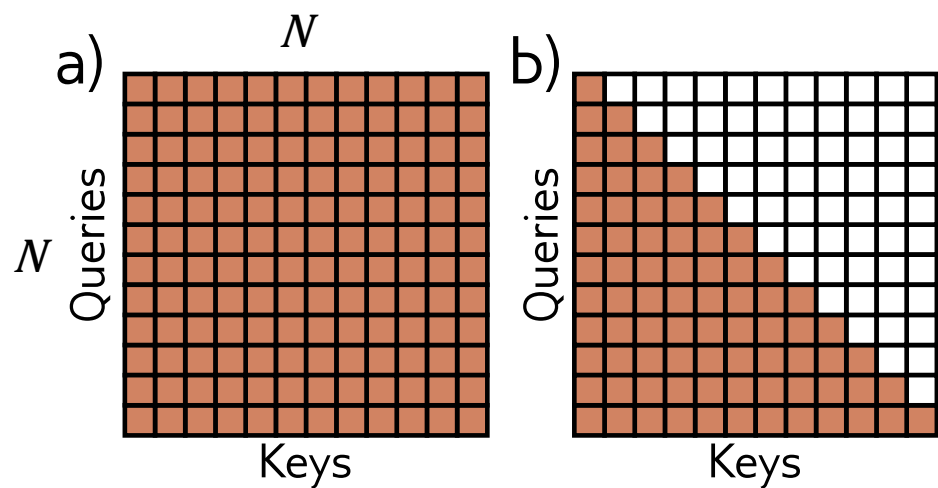
Attention Matrix



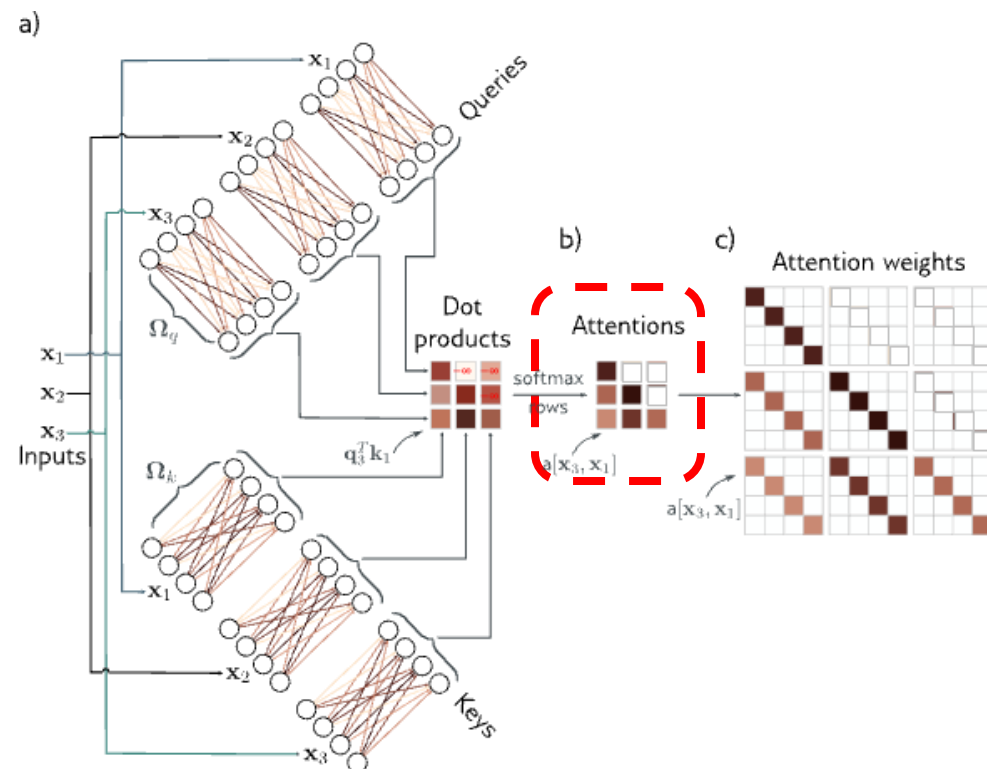
Scales quadratically with sequence length N , e.g. N^2 .



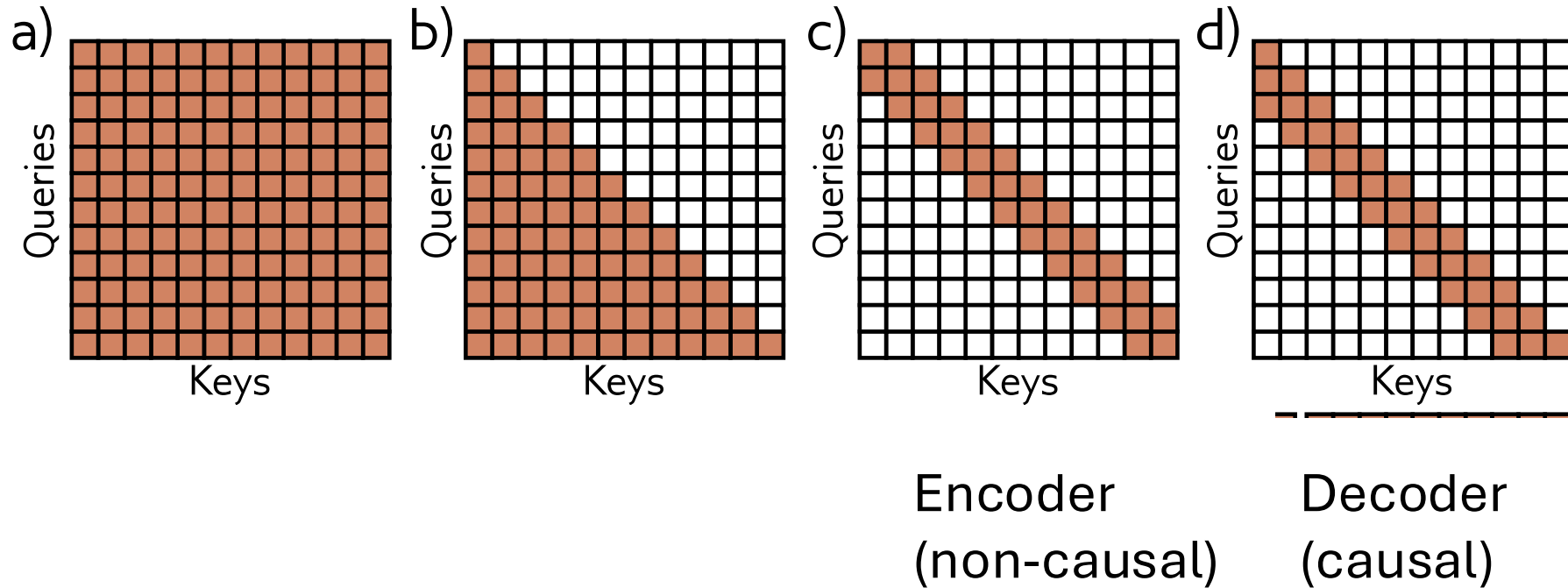
Masked Attention



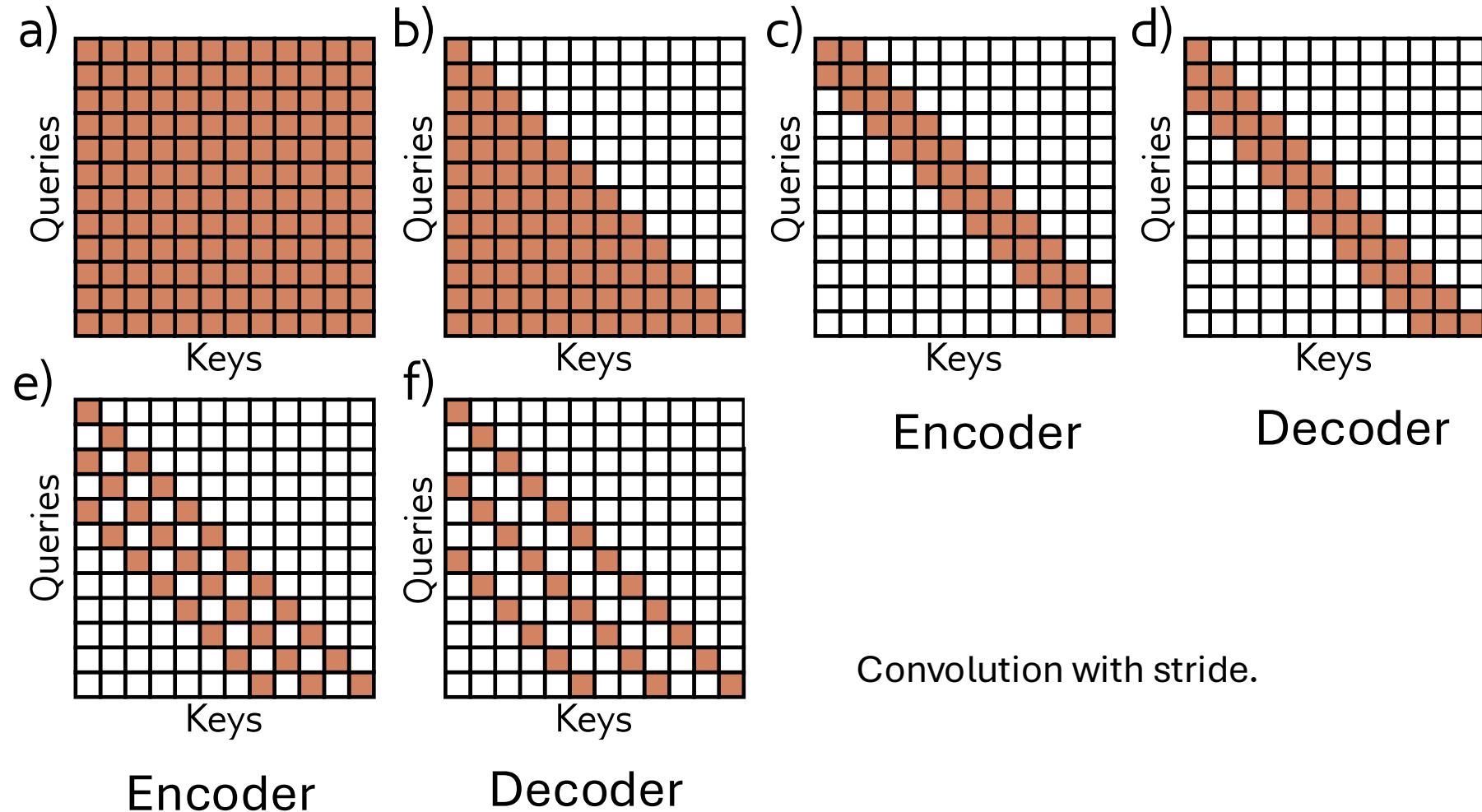
~1/2 the interactions but
still scales quadratically



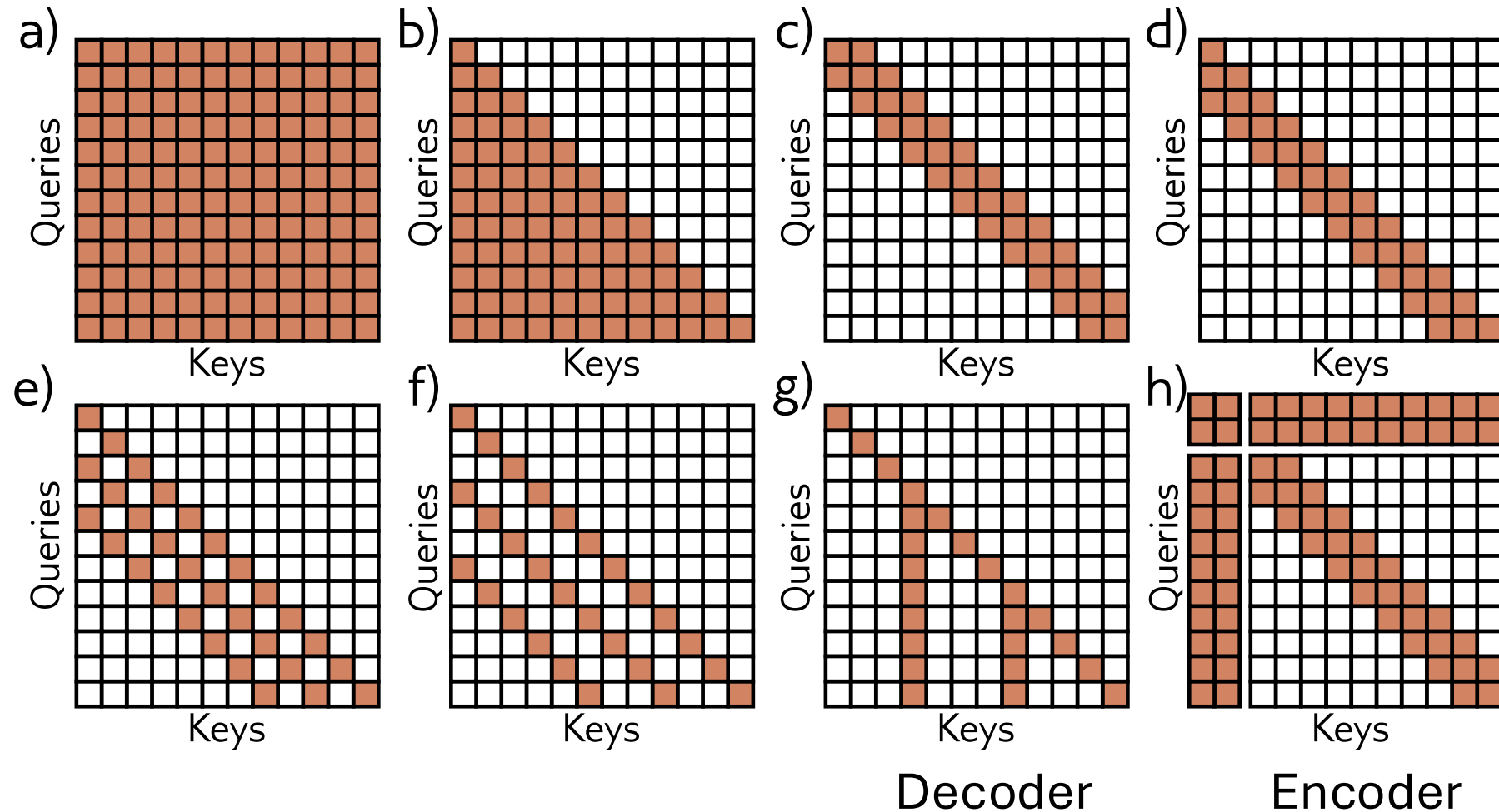
Use Convolutional Structure in Attention



Dilated Convolutional Structures



Have some tokens interact globally



Many of Attempts at Sub-Quadratic Attention

- AFAIK none in state-the-art-models
 - Many published papers claiming linear or $n \log n$ scaling with comparable performance, but none demonstrated at the same scale.
 - 10-20B parameters vs 500B parameters.
- Many practical speedups for leaner quadratic attention.
 - FlashAttention is popular. Same calculations but optimized to be IO-aware.

Any Questions?



Moving on

- Transformer recap
- What are tokens?
- Tokenization and word embedding
- Next token selection
- Transformers for long sequences